

Binary Classification: Credit Scores

Ananya Shah
Ada Wong

Contents

1 Introduction

2 Unsupervised Learning

3 Supervised Learning

Logistic Regression

Support-Vector Machine (SVM)

Neural Network

4 Conclusion



Introduction

Dataset

-  Sourced from OpenML
-  1994 Germany
-  1000 people
-  20 attributes
(numerical and categorical)

Credit History

*credits taken,
paid back duly,
delays, critical
accounts*

Employment

*Number of
years*

Foreign Worker

Yes or no

Personal Status

Married, single...

etc...

Goals

Q: Does a person have good/bad credit risk?



1 Experiment with supervised learning models:
logistic vs SVM vs neural network



2 Extract patterns using unsupervised learning



3 Compare best models to predict good/bad credit risks



Unsupervised Learning

PCA and KMeans

Steps

1. **Perform dimensionality reduction using PCA (n=0.95) and put transformed data into a data frame**
2. **Run KMeans using 3 clusters**
3. **Group data by cluster, and output the mean for each cluster along with each cluster to draw patterns**
4. **Visualize cluster**

```
# explore patterns in data from clustering and dimensionality

# grouping data by clustering
cluster_summary = pca_df.groupby('cluster').mean()
print(cluster_summary)

# seeing how transformed data works alongside original dataset
clustered_data = pd.concat([data, pca_df['cluster']], axis=1)

numerical_clustered_data = clustered_data.select_dtypes(include=['number'])
# analyze cluster characteristics
print(numerical_clustered_data.groupby('cluster').mean())
```

Grouping the data by cluster

Results

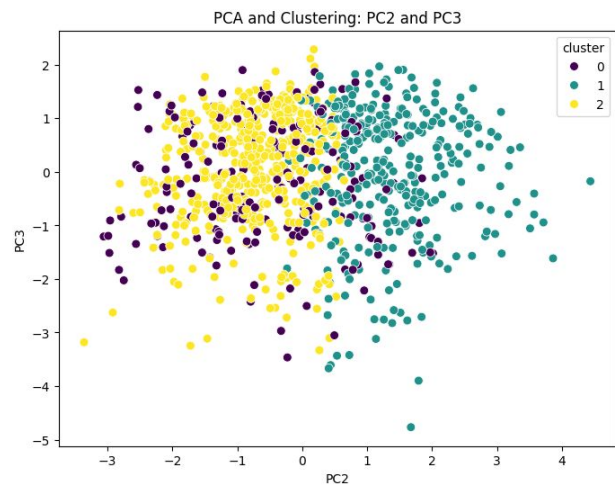
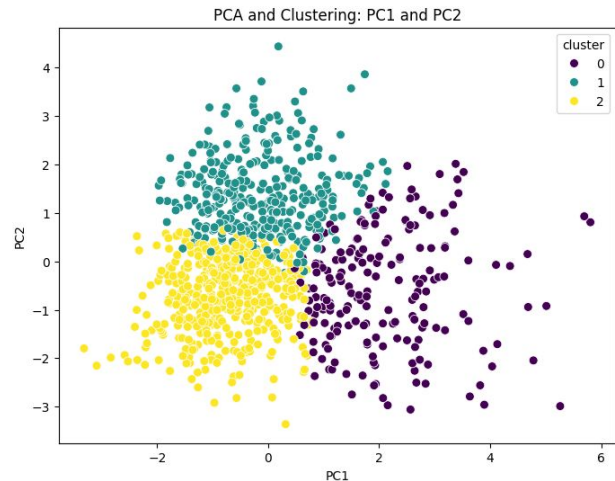
	PC1	PC2	PC3	PC4	PC5	PC6	PC7 \
cluster							
0	2.082902	-0.591127	-0.191216	-0.086041	0.079356	0.180818	0.060597
1	-0.760495	-0.736339	0.104628	-0.152483	-0.027791	-0.088228	-0.052392
2	-0.147303	1.370872	-0.035070	0.262775	-0.007263	0.018212	0.037962

	PC8	PC9	PC10
cluster			
0	-0.231866	-0.117050	-0.047522
1	0.021743	0.041590	0.057635
2	0.104215	0.009878	-0.052868

	checking_status	duration	credit_amount	savings_status \
cluster				
0	1.056410	38.343590	7551.410256	0.974359
1	1.034115	16.304904	2219.654584	1.247335
2	0.922619	17.199405	2255.104167	1.235119

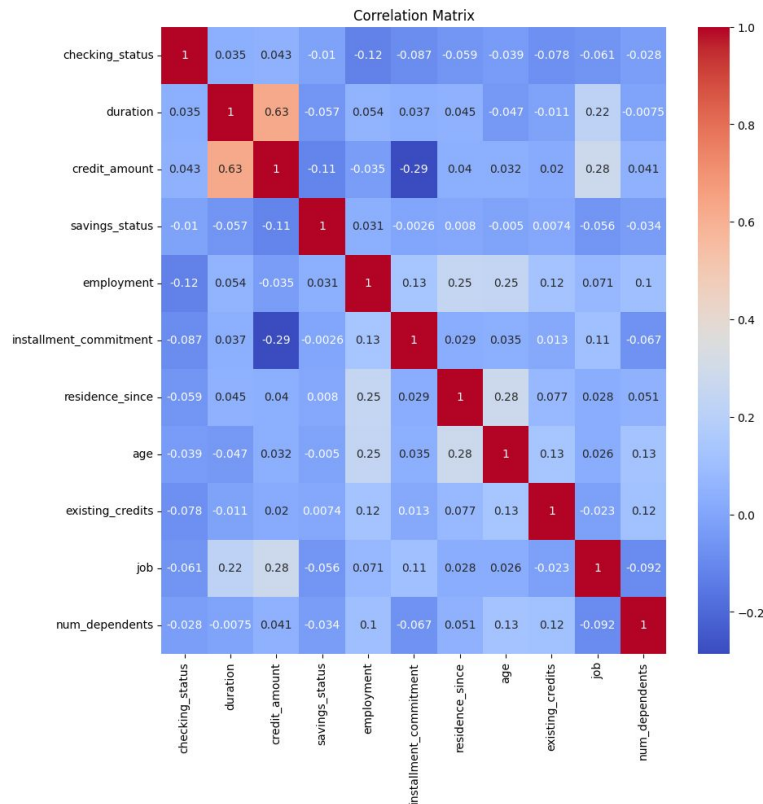
	employment	installment_commitment	residence_since	age \
cluster				
0	2.292308		2.707692	2.861538 35.241026
1	1.812367		2.876333	2.356077 29.349680
2	3.235119		3.261905	3.517857 44.372024

	existing_credits	job	num_dependents	class
cluster				
0	1.456410	2.276923	1.158974	0.548718
1	1.228145	1.793177	1.089552	0.703625
2	1.627976	1.842262	1.244048	0.782738

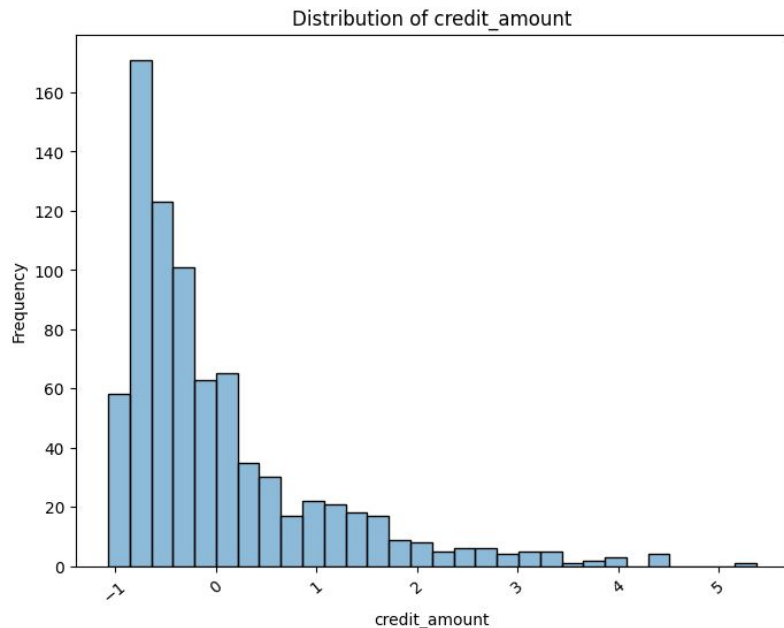


Correlation Matrix

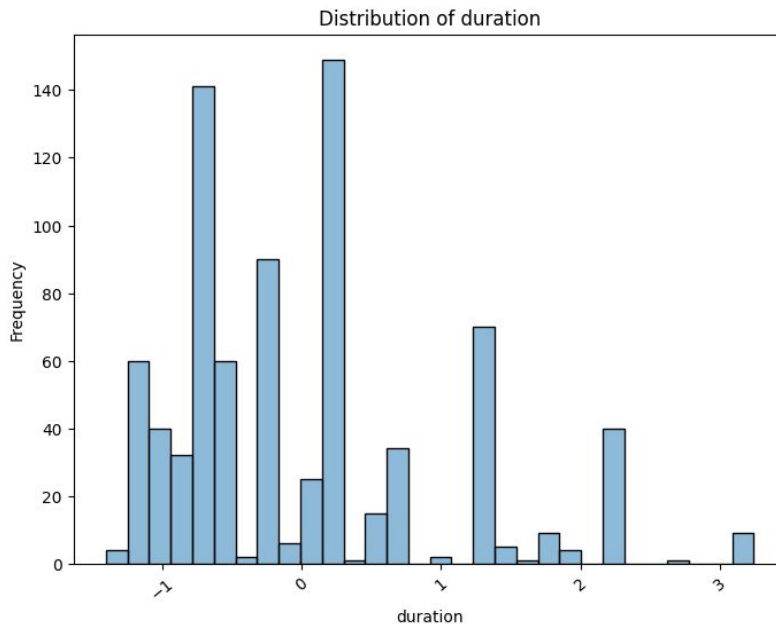
1. **Credit Amount and Duration have a strong positive correlation (0.63)**
2. **Installment Commitment and Credit Amount are negatively correlated (-0.29)**



Individual Features

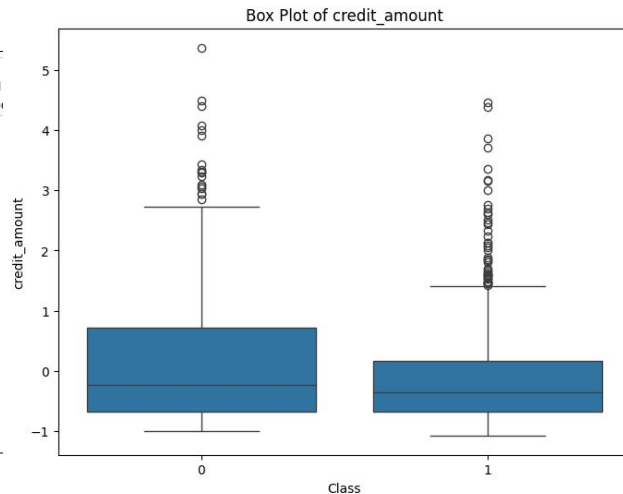
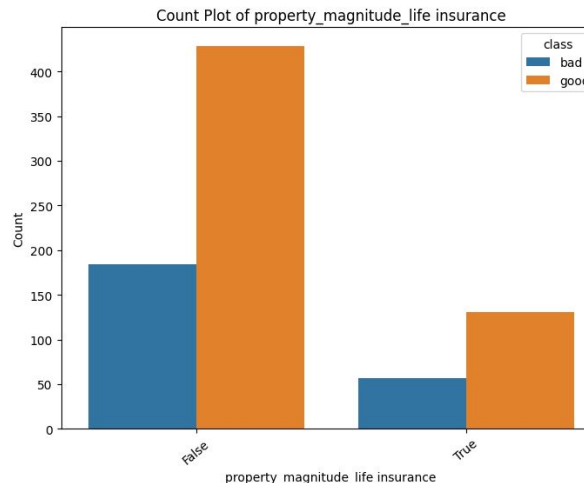
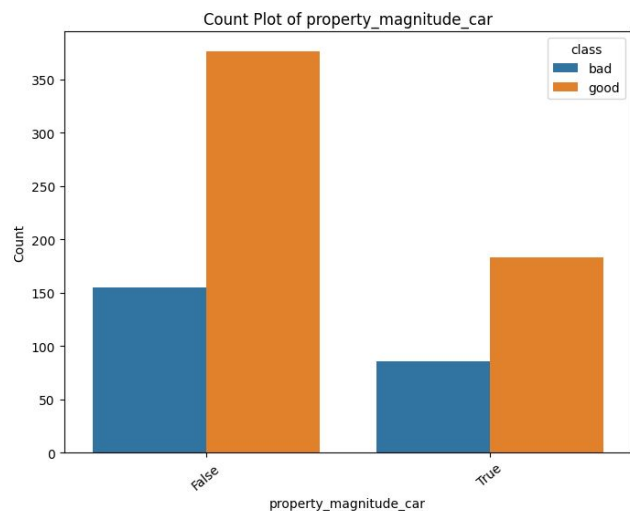


- Credit amount skewed right; majority of people take smaller loans



- Higher frequency for smaller values; short term loans are most common, though there are a few who take longer loans

Feature to Target Variable



- Majority who have life insurance are in the “good” class; owning life insurance generally correlated to having good credit risk
- Majority who own a car are in the “good” class; owning a car generally correlated to having good credit risk
- Classes 0 (“bad”) and 1 (“good”) box plots have similar medians, but varying ranges; Tighter range for class 1 indicates more consistency in credit amount for individuals with good credit risk



Supervised Learning

Pre-processing

Pre-processing

Steps

1. **Data Entries: Byte-string > String**
2. **Encode categorical features**
 - a. Ordinal for those with natural order
 - b. One-encoding for those without natural order
3. **Standardize**
numerical/ordinal features
4. **Split** into 80/10/10 for training, validation, test sets

```
# savings_status order
savings_status_mapping = {
    'no known savings': 0,
    '<100': 1,
    '100<=X<500': 2,
    '500<=X<1000': 3,
    '>=1000': 4
}
df['savings_status'] = df['savings_status'].replace(savings_status_mapping)
```

*Example of One-Hot Encoding:
feature **savings_status***

Logistic

Steps

1) Train Initial Model (use scikit-learn)

```
logreg = LogisticRegression(penalty=None, solver='lbfgs', max_iter=1000)
```

2) Evaluate initial model: confusion matrix, ROC

3) Experiment with Feature Transformation

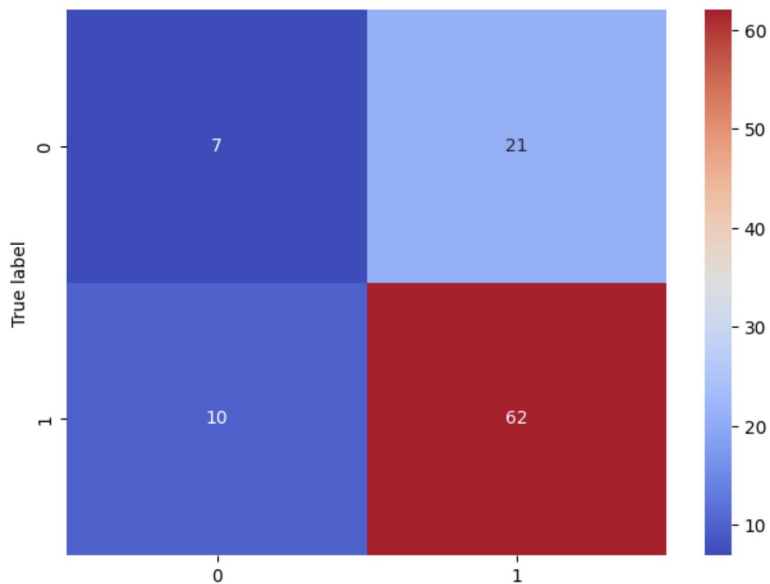
- Polynomial Transform degree 3
- PCA
- RBF-Kernel

4) Experiment with Regularization

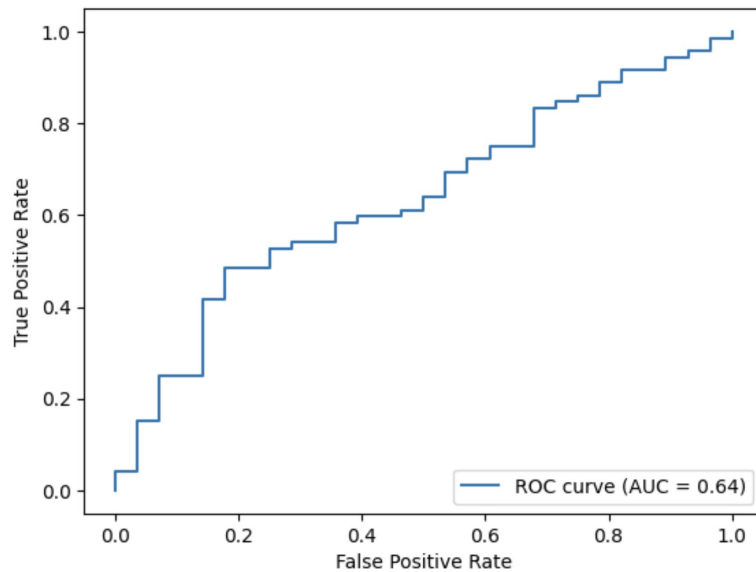
5) Train model with best settings and evaluate on test set

Initial Model Performance

Confusion Matrix (No Regularization) on Test Set



ROC Curve: Test Set



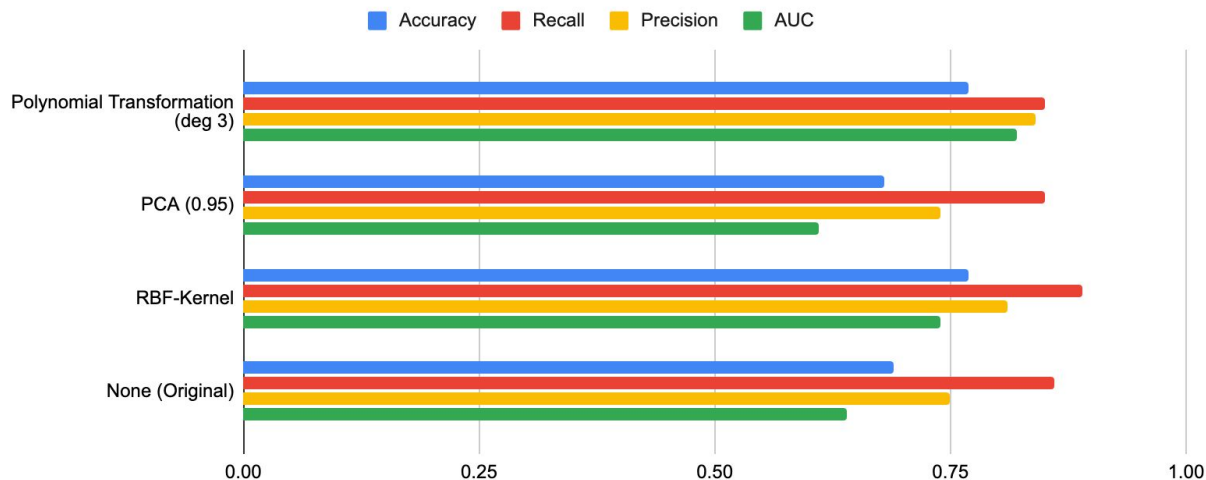
Initial Model Performance

Training Set				Test Set			
Precision	Recall	Accuracy	AUC (ROC Curve)	Precision	Recall	Accuracy	AUC (ROC Curve)
0.79	0.9	0.76	0.8	0.75	0.86	0.69	0.64

- Reasonable in-sample accuracy (0.76)
- Relatively low test accuracy (0.69)
 - Low generalizability to unseen data
- Relatively low AUC (0.64)
 - To be improved...

Feature Transformation

Performance on Validation Set: Feature Transformations



- Judge by Accuracy and AUC (validation set)
- Best: Polynomial Transformation deg 3

Feature Transformation

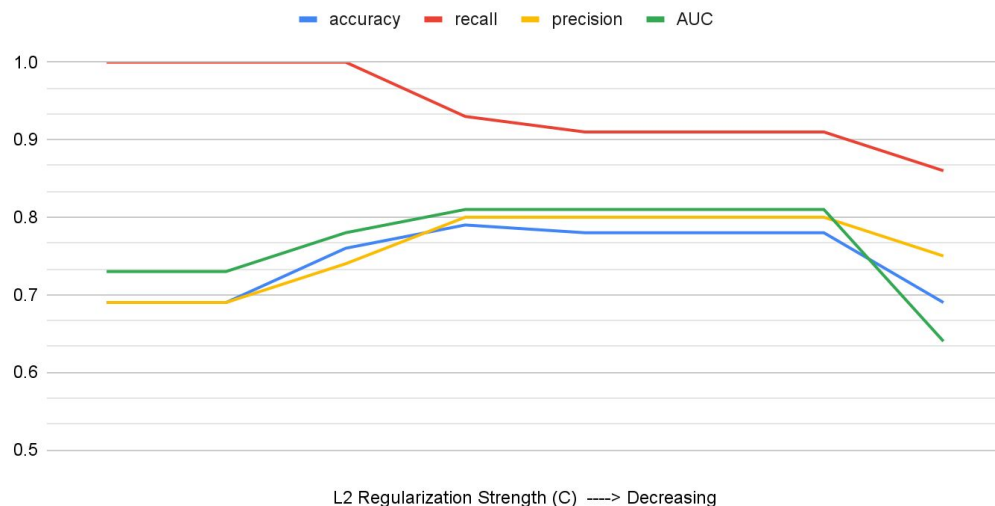
- Suggests there exist **non-linear relationships** in data
 - Benefits from transforming data to higher dimension
- vs PCA:
 - Underfitting (see low in-sample accuracy)
 - Reason: sacrifices # of input features
- vs RBF Kernel:
 - Overfitting (see perfect in-sample accuracy)

	Training Set				Validation Set			
	Precision	Recall	Accuracy	AUC (ROC Curve)	Precision	Recall	Accuracy	AUC (ROC Curve)
Polynomial Transformation (deg 3)	1	1	1	1	0.84	0.85	0.77	0.82
PCA (0.95)	0.76	0.91	0.74	0.77	0.74	0.85	0.68	0.61
RBF-Kernel	1	1	1	1	0.81	0.89	0.77	0.74
None (Original)	0.79	0.9	0.76	0.8	0.75	0.86	0.69	0.64

Regularization (L2)

Performance on Validation Set: Different L2 Regularization Strengths

No feature transformation



- **Best: $C = 1$**
- **Too much regularization** (left of graph)
 - Lower accuracy and AUC
 - Reason: Fails to capture complex relationship during training
- **Too little regularization** (right of graph)
 - No improvement - plateau

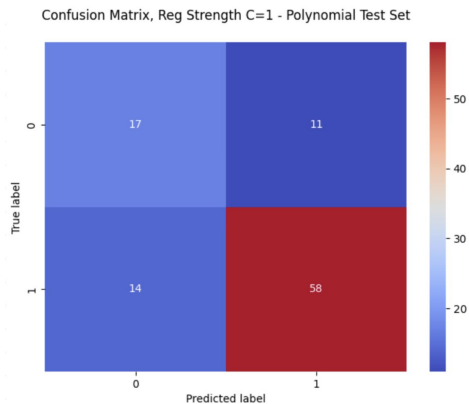
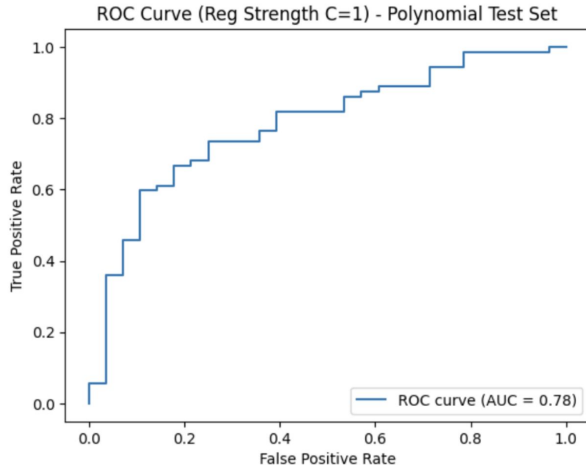
Best Logistic Model

Final Test Accuracy = 75%
(↑ by 6%)

Hyperparameters:

- **Reg Strength = 1**
- **Feature Transformation = Polynomial degree 3**

Test Set			
Precision	Recall	Accuracy	AUC (ROC Curve)
0.84	0.81	0.75	0.78



SVM

Initial Model Performance

Training Set					Validation Set					Test Set				
Precision	Recall	Accuracy	Error (%)	Hinge Loss	Precision	Recall	Accuracy	Error (%)	Hinge Loss	Precision	Recall	Accuracy	Error (%)	Hinge Loss
0.78	0.9	0.76	24	0.55	0.79	0.96	0.8	20	0.51	0.74	0.89	0.7	30	0.58

- Better accuracy on training and validation set - slight overfitting
- Reasonably high test set accuracy of 70% - generally good performance
- Hinge Loss of 0.58 - reasonably high

Feature Transformations

Training Set					Validation Set					Test Set				
Precision	Recall	Accuracy	Error (%)	Hinge Loss	Precision	Recall	Accuracy	Error (%)	Hinge Loss	Precision	Recall	Accuracy	Error (%)	Hinge Loss
0.78	0.9	0.76	24	0.55	0.79	0.96	0.8	20	0.51	0.74	0.89	0.7	30	0.58
0.86	0.98	0.88	12	0.42	0.79	0.96	0.8	20	0.51	0.78	0.94	0.77	23	0.51
0.83	0.99	0.85	15	0.45	0.79	0.97	0.8	20	0.51	0.73	0.92	0.7	30	0.58
0.83	0.99	0.85	15	0.44	0.78	0.97	0.79	21	0.52	0.74	0.9	0.7	30	0.58

- Row 1: Original
- Row 2: Polynomial Kernel of Degree 3
- Row 3: PCA Transformation
- Row 4: RBF Kernel

Hyperparameter Tuning

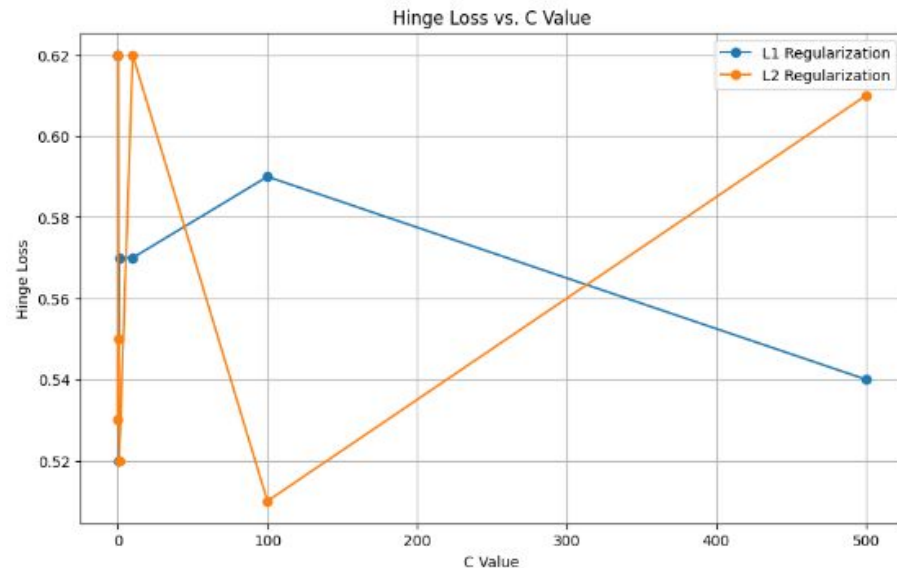
- Two regularization techniques: L1 and L2 Regularization
- Hyperparameters: Learning Rate, Regularization Strength (C)
- Best hyperparameters to yield lowest error on validation set and lowest hinge loss

Results

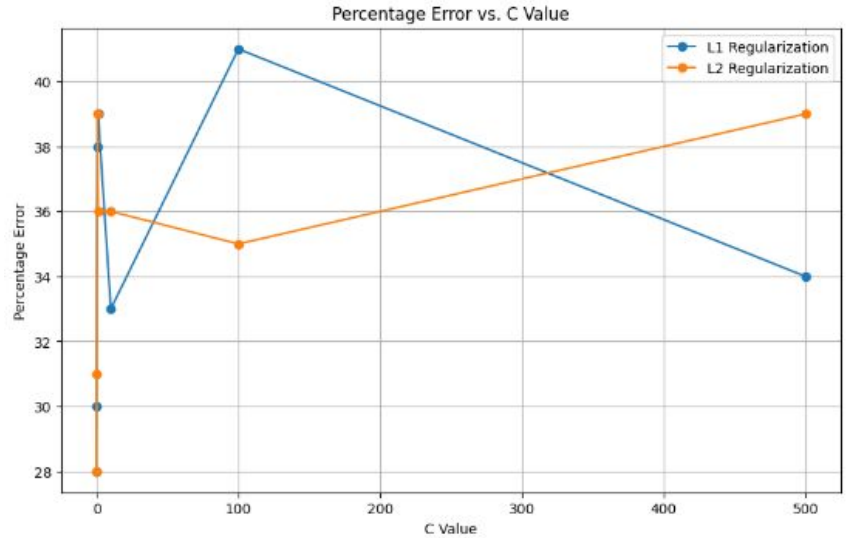
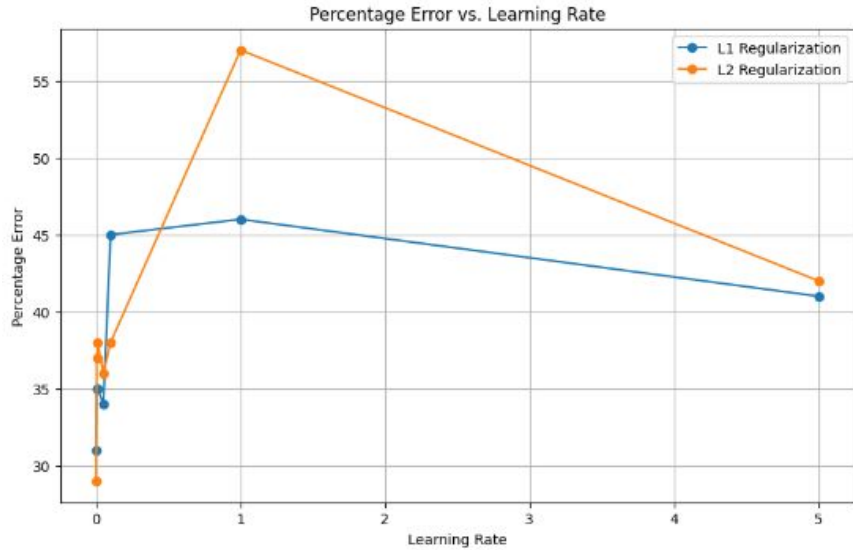
L1 Regularization:		
Best Hyperparameters:	Lowest Hinge Loss: 0.51	Percentage Error: 28%
Learning Rate:	0.1	0.001
Regularization Strength (C):	100	0.01
Precision:	0.78	1
Recall:	0.73	0.72

L2 Regularization:		
Best Hyperparameters:	Lowest Hinge Loss: 0.51	Percentage Error: 28%
Learning Rate:	0.05	0.001
Regularization Strength (C):	10	0.01
Precision:	0.85	1
Recall:	0.73	0.72

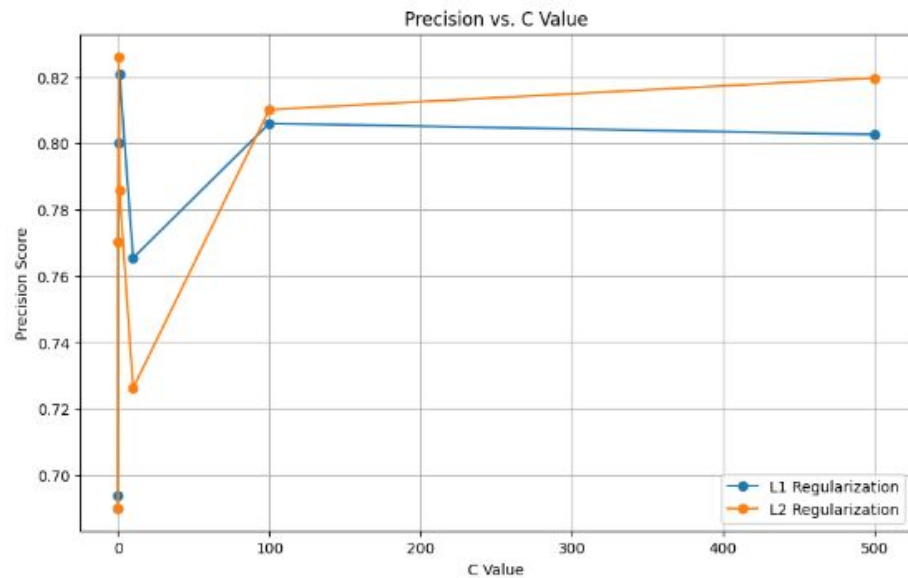
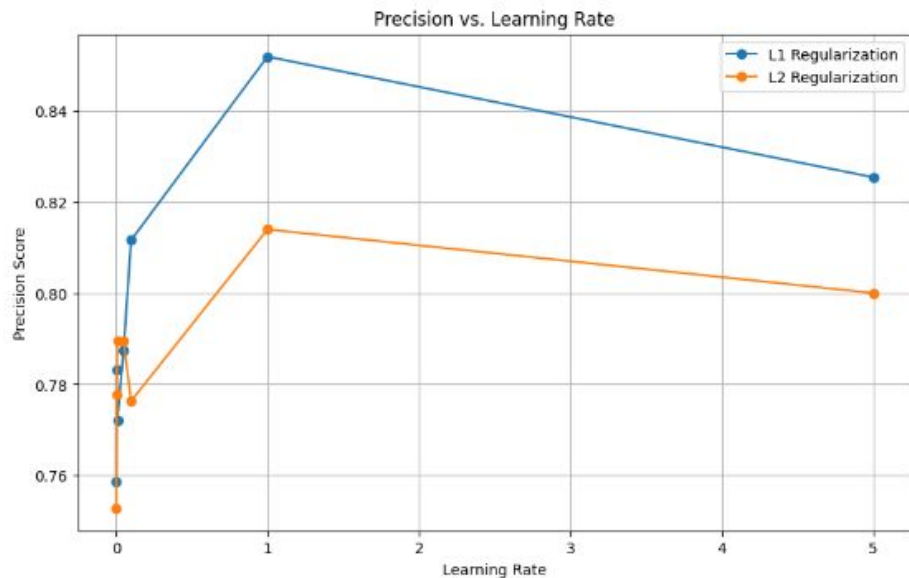
Hinge Loss



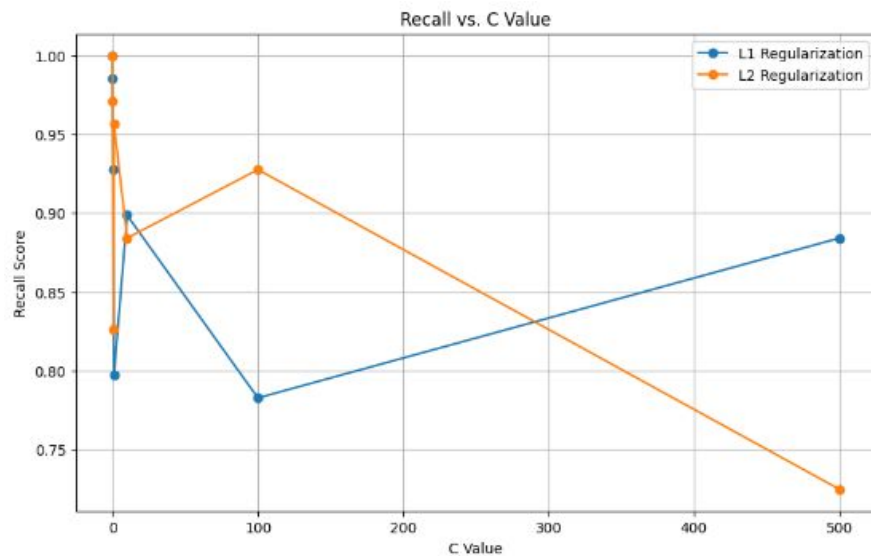
Accuracy/Error



Precision



Recall



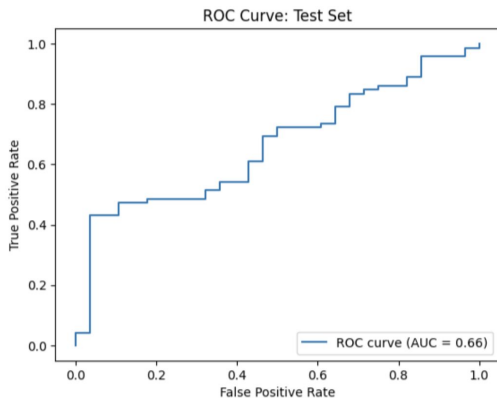
Key Findings

- Using a polynomial kernel of degree 3 generated the best performance (accuracy of 77%)
- The model benefitted most with small values for hyperparameters: a learning rate of 0.001, a regularization strength of C
- The model originally had a reasonably high test set accuracy of 70%; too much regularization would lead to too simple of a model, and higher risk of underfitting
- To maximize precision, a small learning rate and large value of C is ideal; To maximize recall, a large learning rate and small value of C is ideal

Neural Network

Initial Model Performance

Training Set				Test Set			
Precision	Recall	Accuracy	AUC (ROC Curve)	Precision	Recall	Accuracy	AUC (ROC Curve)
0.81	0.9	0.78	0.82	0.74	0.85	0.68	0.66



- 2 hidden layers
- No regularization
- Test Accuracy = 68%

Hidden Layers

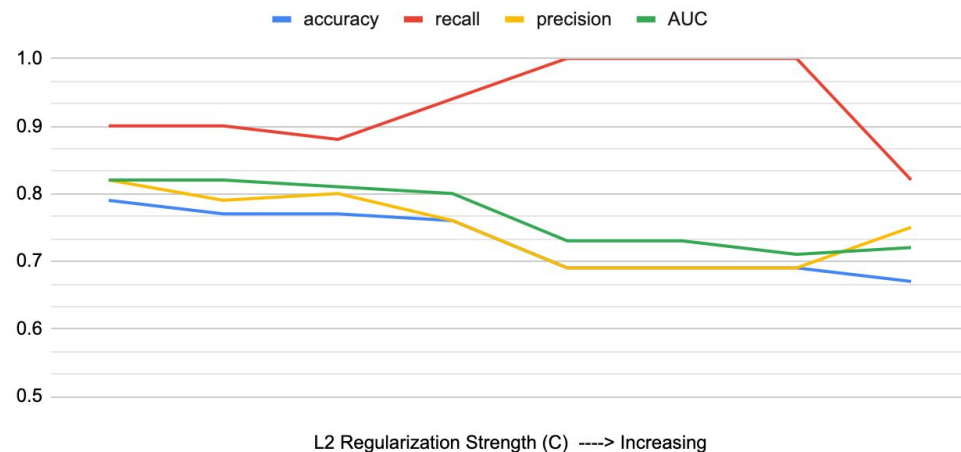
	Training Set				Validation Set			
	Precision	Recall	Accuracy	AUC (ROC Curve)	Precision	Recall	Accuracy	AUC (ROC Curve)
2 (Original)	0.81	0.9	0.78	0.82	0.74	0.85	0.68	0.66
10	0.83	0.92	0.83	0.88	0.81	0.87	0.77	0.83
15	0.87	0.94	0.86	0.92	0.82	0.87	0.78	0.78
30	0.92	0.96	0.91	0.96	0.85	0.87	0.8	0.79

- **Best: 10 hidden layers**
 - Highest AUC on validation set
 - Best balance for bias-variance:
 - Enough hidden layers to capture complexity
 - Not too many layers to overfit training data

Regularization (L2)

Performance on Validation Set: Different L2 Regularization Strengths

No feature transformation

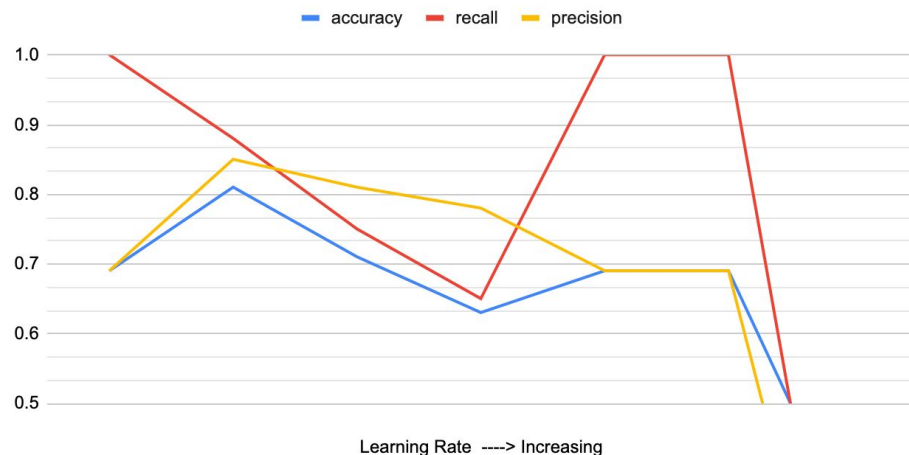


- **Best: $C = 10^{-6}$** (least strength; left of graph)
 - Highest validation accuracy
- No need to apply regularization
- Suggests data is not overfitted to training data

Learning Rate

Performance on Validation Set: Different Learning Rates

No feature transformation



- **Compared:** $10^{-5}, 10^{-3}, 10^{-1}, 10, 10^3, 10^5, 10^7$
- **Best: $C = 10^{-3}$**
 - Highest validation accuracy
- Learning rate too small:
 - Updates insufficient to reach optimal parameters during gradient descent
- Learning rate too large:
 - Overshoot optimum during gradient descent

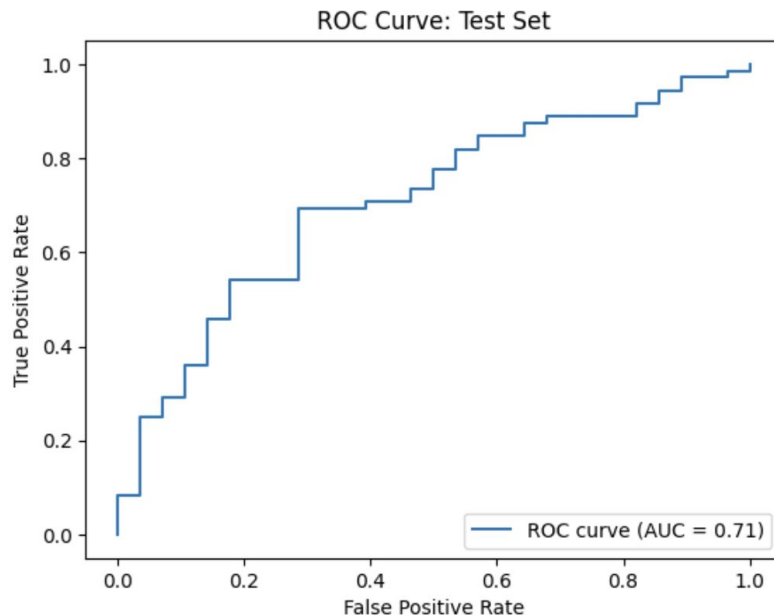
Best Neural Network

Final Test Accuracy = 71%
(↑ by 3%)

Hyperparameters:

- 10 hidden layers
- Reg Strength = 10^{-6}
- Learning Rate = 0.001

Test Set			
Precision	Recall	Accuracy	AUC (ROC Curve)
0.77	0.85	0.71	0.71





Conclusion

Final Takeaways

- Best Overall Model: SVM
 - Accuracy of 77%
 - Polynomial kernel of degree 3
- The dataset's performance benefited most with small regularization, and for some models, no regularization